Ejercicio 1: Despertador con snooze (`while`)

Enunciado: crea `dormir(int pendientes)`, donde `pendientes` son los minutos que faltan para levantarse. Mientras `pendientes` sea mayor que 0, el método suma un snooze y resta 5 minutos. En cada vuelta imprime `Snooze N: faltan X min`. Al terminar imprime `Me levanto tras N snoozes.` Si no hubo ningún snooze, imprime `Ya estaba despierto, qué sospechoso.` Llámalo con 20 y con 0.

java

```
public class Main { // clase principal

    static void dormir(int pendientes) { // recibe los minutos pendientes

        int snooze = 0; // contador de snoozes pulsados

        while (pendientes > 0) { // se repite mientras queden minutos

            snooze++; // pulsamos un snooze más

            pendientes -= 5; // cada snooze resta 5 minutos

            System.out.println("Snooze " + snooze + ": faltan " + pendientes + "
min"); // informe de la vuelta

        } // cierra el while

        if (snooze == 0) { // caso límite: el bucle no llegó a ejecutarse

            System.out.println("Ya estaba despierto, qué sospechoso."); // mensaje
especial

        } else { // si hubo al menos un snooze

            System.out.println("Me levanto tras " + snooze + " snoozes."); //
mensaje final

        } // cierra el if-else

    } // cierra dormir


    public static void main(String[] args) { // punto de entrada

        System.out.println("Lunes:"); // título del lunes

        dormir(20); // 20 minutos pendientes
```

PROMETEO
by thePower

```
        System.out.println("Martes:"); // título del martes
        dormir(0); // nada pendiente
    } // cierra main
} // cierra la clase
```

text

```
Lunes:
Snooze 1: faltan 15 min
Snooze 2: faltan 10 min
Snooze 3: faltan 5 min
Snooze 4: faltan 0 min
Me levanto tras 4 snoozes.
Martes:
Ya estaba despierto, qué sospechoso.
```

Explicación: `while` comprueba la condición **antes** de cada vuelta. Por eso, si la condición es falsa desde el principio, el cuerpo no se ejecuta ni una vez, como pasa con `dormir(0)`. Dentro del bucle hay que modificar la variable de la condición (`pendientes -= 5`). Si no, el bucle sería infinito. El contador `snooze` permite saber después si el bucle llegó a ejecutarse.

Ejercicio 2: Cena de croquetas (`do-while`)

Enunciado: crea `cenar(int croquetas)`. El método usa un `do-while`, así que el cuerpo se ejecuta siempre al menos una vez. Si queda alguna croqueta, se come una y se imprime `Croqueta comida, quedan N`. Si no queda ninguna, imprime `No quedaba ninguna, pero el intento cuenta.` El bucle se repite mientras queden croquetas. Llámalo con 3 y con 0.

java

```
public class Main { // clase principal
    static void cenar(int croquetas) { // recibe las croquetas disponibles
        do { // el cuerpo se ejecuta al menos una vez
            if (croquetas > 0) { // si queda alguna croqueta
```

```
                croquetas--; // nos comemos una

                System.out.println("Croqueta comida, quedan " + croquetas); //
informamos de cuántas quedan

            } else { // si no queda ninguna

                System.out.println("No quedaba ninguna, pero el intento cuenta.");
// mensaje del primer intento fallido

            } // cierra el if-else

        } while (croquetas > 0); // repetimos mientras queden croquetas

    } // cierra cenar


    public static void main(String[] args) { // punto de entrada

        System.out.println("Cena 1:"); // título de la cena 1

        cenar(3); // cena con 3 croquetas

        System.out.println("Cena 2:"); // título de la cena 2

        cenar(0); // cena sin croquetas

    } // cierra main

} // cierra la clase
```

text

```
Cena 1:

Croqueta comida, quedan 2

Croqueta comida, quedan 1

Croqueta comida, quedan 0

Cena 2:

No quedaba ninguna, pero el intento cuenta.
```

Explicación: `do-while` comprueba la condición **después** de ejecutar el cuerpo, así que siempre da al menos una vuelta. Esa es la diferencia clave con `while`. Con 0 croquetas, el cuerpo se ejecuta una vez y la condición `croquetas > 0` ya es falsa. Al final de `do { ... } while (condición);` hay un punto y coma obligatorio.

Ejercicio 3: El mando desaparecido (`break`)

Enunciado: crea `buscar(String[] sitios, String objetivo)`. Recorre el array con un `for` e imprime `Mirando: sitio` en cada posición. Si el sitio es igual al objetivo (usa `equals`), guarda su posición y sal del bucle con `break`. Después del bucle, si hubo éxito imprime `¡Mando encontrado en X (posición P)!` Si no, imprime `El mando sigue desaparecido tras mirar N sitios.` Los sitios son `{"sofá", "nevera", "cojín", "microondas"}`. Busca `"nevera"` y `"lavadora"`.

java

```
public class Main { // clase principal

    static void buscar(String[] sitios, String objetivo) { // recibe los sitios y el sitio buscado

        int pos = -1; // -1 significa "todavía no encontrado"

        for (int i = 0; i < sitios.length; i++) { // recorremos los sitios

            System.out.println("Mirando: " + sitios[i]); // anunciamos el sitio actual

            if (sitios[i].equals(objetivo)) { // si es el sitio buscado

                pos = i; // guardamos la posición

                break; // salimos del for inmediatamente

            } // cierra el if

        } // cierra el for

        if (pos >= 0) { // si pos cambió, lo encontramos

            System.out.println("¡Mando encontrado en " + objetivo + " (posición " + pos + ")!"); // éxito

        } else { // si pos sigue en -1

            System.out.println("El mando sigue desaparecido tras mirar " + sitios.length + " sitios."); // fracaso

        } // cierra el if-else

    } // cierra buscar


    public static void main(String[] args) { // punto de entrada
```

```
        String[] sitios = {"sofá", "nevera", "cojín", "microondas"}; // lugares
donde mirar

        System.out.println("Búsqueda 1:"); // título 1

        buscar(sitios, "nevera"); // buscamos en la nevera

        System.out.println("Búsqueda 2:"); // título 2

        buscar(sitios, "lavadora"); // buscamos en un sitio que no está

    } // cierra main

} // cierra la clase
```

text

```
Búsqueda 1:

Mirando: sofá

Mirando: nevera

¡Mando encontrado en nevera (posición 1)!

Búsqueda 2:

Mirando: sofá

Mirando: nevera

Mirando: cojín

Mirando: microondas

El mando sigue desaparecido tras mirar 4 sitios.
```

Explicación: `break` termina de golpe el bucle más interno que lo contiene. Ahorra vueltas cuando ya se tiene lo buscado, como se ve en la búsqueda 1, que no llega a mirar el cojín. Un valor centinela como `-1` permite distinguir después del bucle entre "encontrado" y "no encontrado". Un bucle que acaba sin `break` simplemente recorrió todo el array.

Ejercicio 4: Compra con tentaciones (`continue`)

Enunciado: crea `comprar(String[] lista)`. Recorre la lista con un `for`. Si el producto es `"gominolas"`, súmalo a las confiscadas y pasa al siguiente con `continue`. El resto se imprime como `Comprado: producto`. Al final, si no se compró nada, imprime `Carrito vacío: solo había tentaciones.` Después imprime siempre `Gominolas confiscadas: N`. Pruébalo con `{"pan", "gominolas", "leche", "gominolas", "huevos"}` y con `{"gominolas"}`.

java

```
public class Main { // clase principal

    static void comprar(String[] lista) { // recibe la lista de la compra

        int comprados = 0; // productos que sí entran en el carrito

        int confiscadas = 0; // gominolas retiradas

        for (int i = 0; i < lista.length; i++) { // recorremos la lista

            if (lista[i].equals("gominolas")) { // si es una tentación

                confiscadas++; // la contamos como confiscada

                continue; // saltamos al siguiente producto

            } // cierra el if

            System.out.println("Comprado: " + lista[i]); // esta línea solo se
alcanza si no era gominola

            comprados++; // sumamos un producto comprado

        } // cierra el for

        if (comprados == 0) { // caso límite: no se compró nada

            System.out.println("Carrito vacío: solo había tentaciones."); // aviso

        } // cierra el if

        System.out.println("Gominolas confiscadas: " + confiscadas); // resumen
final

    } // cierra comprar



    public static void main(String[] args) { // punto de entrada

        String[] lista1 = {"pan", "gominolas", "leche", "gominolas", "huevos"}; //
lista con 2 tentaciones

        String[] lista2 = {"gominolas"}; // lista solo de tentaciones
```

```
        System.out.println("Lista 1:"); // título 1
        comprar(lista1); // procesamos la lista 1
        System.out.println("Lista 2:"); // título 2
        comprar(lista2); // procesamos la lista 2
    } // cierra main
} // cierra la clase
```

text

```
Lista 1:
Comprado: pan
Comprado: leche
Comprado: huevos
Gominolas confiscadas: 2
Lista 2:
Carrito vacío: solo había tentaciones.
Gominolas confiscadas: 1
```

PROMETEO
by thePower

Explicación: `continue` no sale del bucle como `break`. Solo abandona la vuelta actual y salta a la siguiente. En un `for`, esa siguiente vuelta incluye el `i++`, así que el índice sigue avanzando. Todo lo que está después del `continue` en el cuerpo se omite en esa vuelta. En este ejercicio las gominolas se cuentan antes de saltar.

Ejercicio 5: Traductor de ladridos (`switch` con `return`)

Enunciado: crea `static String traducir(String ladrido)`. Usa un `switch` sobre el texto con estas reglas:

- `"guau"` devuelve `Hola, humano`.
- `"grrr"` devuelve `Aléjate de mi hueso`.
- `"miau"` devuelve `Dame comida ya`.
- El texto vacío `""` devuelve `Silencio sospechoso`.
- Cualquier otro valor (`default`) devuelve `Idioma desconocido`.

Cada `case` termina con `return`. En `main`, recorre el array `{"guau", "grrr", "", "quack", "miau"}` con un `for`. Imprime cada línea con el formato `Ladrido 'X' = traducción`.

java

```
public class Main { // clase principal

    static String traducir(String ladrido) { // devuelve la traducción de un sonido

        switch (ladrido) { // elegimos rama según el valor del texto

            case "guau": // si el sonido es "guau"

                return "Hola, humano"; // return sale del método, no hace falta
break

            case "grrr": // si el sonido es "grrr"

                return "Aléjate de mi hueso"; // traducción de grrr

            case "miau": // si el sonido es "miau"

                return "Dame comida ya"; // traducción de miau

            case "": // caso límite: texto vacío

                return "Silencio sospechoso"; // traducción del silencio

            default: // cualquier otro valor

                return "Idioma desconocido"; // traducción por defecto

        } // cierra el switch

    } // cierra traducir


    public static void main(String[] args) { // punto de entrada

        String[] ladridos = {"guau", "grrr", "", "quack", "miau"}; // sonidos a
traducir

        for (int i = 0; i < ladridos.length; i++) { // recorremos el array

            System.out.println("Ladrido '" + ladridos[i] + "' = " +
traducir(ladridos[i])); // imprimimos sonido y traducción

        } // cierra el for

    } // cierra main

} // cierra la clase
```

text

```
Ladrido 'guau' = Hola, humano

Ladrido 'grrr' = Aléjate de mi hueso

Ladrido '' = Silencio sospechoso

Ladrido 'quack' = Idioma desconocido

Ladrido 'miau' = Dame comida ya
```

Explicación: `switch` admite textos desde Java 7 y compara con `equals`. Un `case` sin `break` ni `return` sigue ejecutando el siguiente (*fall-through*). Aquí `return` evita ese problema, porque termina el método en cuanto se cumple un caso. `default` recoge todo lo que no coincide con ningún `case`, y conviene ponerlo siempre. Java exige que todos los caminos del método devuelvan algo, y con el `default` eso queda garantizado.

PROMETEO
by thePower